

Asset Management

A Codebase Tutorial

For readers brand-new to HTML, CSS, JavaScript, and Node.js

This document walks through every part of the asset-management web application — file by file, concept by concept. We assume nothing. If you have never seen a `<div>` tag, opened a browser developer console, or run a Node.js script, you can still follow along.

We will start with the absolute basics (what HTML, CSS and JavaScript actually are, and how a web server fits in), then explore the project from the outside in: the user-facing pages, the JavaScript that powers them, the server that serves them, the database that stores everything, and the security & AI features that make the application interesting.

Read it linearly the first time. After that, treat it as a reference — every file has its own section, and the table of contents on the next page will get you there in one jump.

Version 3.0.2 • Document generated automatically from source

Table of contents

Part I — The basics, in plain words	3
What is a web app?	3
HTML — the skeleton	3
CSS — the appearance	3
JavaScript — the behaviour	4
Node.js — JavaScript on the server	4
PostgreSQL — the database	4
npm — the package manager	5
The big picture, in one paragraph	5
Part II — Map of the project	6
Top-level layout	6
Two halves: client vs server	6
What is not here	7
Part III — The front end, page by page	8
login.html — the door	8
js/login.js — the form handler	8
register.html + js/register.js — sign-up + payments	8
forgot-password.html + js/forgot-password.js	9
index.html — the dashboard	9
js/app.js — the dashboard's brain (5,610 lines)	11
js/chat.js — the floating AI advisor (675 lines)	13
js/i18n.js — translations (1,321 lines)	13
js/csrf.js — the security gate for write requests (42 lines)	14
Part IV — The back end, file by file	15
config.js — the gatekeeper for secrets (73 lines)	15
db/pool.js — the database connection pool (48 lines)	15
db/schema.sql — the table definitions (132 lines)	16
db/queries.js — every SQL query in the app (1,348 lines)	17
db/migrate-*.sql — incremental schema migrations	18
db/migrate-json-to-pg.js — the one-shot upgrade (259 lines)	18
rotate-encryption-key.js — re-encrypt everything (206 lines)	19
server.js — the monolith (6,083 lines)	19
server.js — the routes	22
Part V — Cross-cutting concepts	25
Authentication, in full	25
CSRF protection, in full	25
Encryption at rest	25
Couples — how shared data flows	26
AI chat — the loop	26
Rate limiting and abuse	27
PDF processing pipeline	27
Part VI — End-to-end walkthroughs	28
Walkthrough 1: "Add a new expense"	28
Walkthrough 2: "Ask the AI: how much did I spend on food in March?"	28
Walkthrough 3: "Set a \$300 monthly budget for transport"	29
Part VII — How to run, deploy, and maintain the app	30
Local development	30
Production	30
Updating the version	30
Rotating the encryption key	30
Reading the logs	31
Part VIII — A glossary of every jargon term used	32
Where to go next	35

Part I — The basics, in plain words

If you already know HTML, CSS, JS and Node, skim this and skip ahead.

Modern web applications combine four very different technologies. Before we open a single project file, here is what each one actually is — using analogies a non-programmer can hold onto.

What is a web app?

A web app is just two computers having a structured conversation. Your **browser** (the client) asks for a page; somebody else's computer (the **server**) sends one back. The page is plain text — describing what to draw, what fonts to use, and what to do when you click a button. The browser reads that text and renders pixels onto your screen.

Everything in this repository is either text the browser will read, text the server will run, or instructions for the database that lives behind the server.

HTML — the skeleton

HTML stands for *HyperText Markup Language*. It is **not a programming language** — it is a way of labelling content so the browser knows what kind of thing each piece is: this chunk is a heading, that one is a paragraph, this one is a button.

Every HTML element is a **tag** wrapped in angle brackets. A tag normally has an opening and a closing form:

```
<h1>Asset Management Dashboard</h1>
<p>Welcome back, Frederico.</p>
<button id="logoutBtn">Log out</button>
```

HTML

Tags can be **nested** — buttons go inside forms, paragraphs go inside sections, and so on. Together they form a tree the browser calls the **DOM** (Document Object Model). When JavaScript wants to change the page, it walks this tree and edits the bits it cares about.

You will see four HTML files in this project: `index.html`, `login.html`, `register.html`, `forgot-password.html`. Each one is a separate "page", but as you will see in Part III, almost all of the heavy lifting on the dashboard happens inside a single big `index.html`.

CSS — the appearance

CSS (*Cascading Style Sheets*) tells the browser **what things should look like**. Colours, spacing, fonts, animations — all CSS. HTML says "this is a button"; CSS says "buttons are terracotta-orange, rounded, and grow slightly when you hover them".

```
button {
  background: #B8593A;
  color: white;
  padding: 8px 16px;
  border-radius: 8px;
}
button:hover { transform: scale(1.02); }
```

CSS

In this project, almost all CSS lives **inside** the HTML files (in a big `<style>` block) instead of in a separate `.css` file. That is a deliberate choice: by keeping the styles together with the markup that uses them, the entire dashboard loads in a single network request — there is no build step that bundles separate files together.

CSS variables

Lines like `--bg: #F3ECE0;` near the top of `index.html` are called CSS custom properties (variables). Later rules reference them with `var(--bg)`. Changing one variable retunes the whole palette — that is how the dark/light/earthy theme switch works (`data-theme="dark"` simply overrides those variables).

JavaScript — the behaviour

JavaScript (JS) is the **only programming language web browsers can run natively**.

Everything dynamic — clicking buttons, fetching data, drawing charts, talking to the server in the background — is JavaScript.

You will see JS in three different places:

- Inside the browser, in files under `js/` such as `js/app.js` and `js/chat.js`. These run on the user's laptop or phone.
- On the server, in `server.js`, `config.js` and `db/*.js`. These run on the machine that hosts the website. Same language, very different role.
- Inline inside HTML — small `<script>` blocks at the top of each HTML file that run before the rest of the page paints (we use them to apply the saved theme so the page never "flashes" the wrong colour).

A function in JavaScript looks like this:

```
function categoryColor(slug) {  
  const c = _userCategoriesBySlug.get(slug);  
  return c ? c.color : ORPHAN_CATEGORY_COLOR;  
}
```

JS

Read that as: "given a category *slug* (a short identifier like `food`), look it up in our internal Map, return the colour stored on it; if we cannot find it, fall back to the orphan colour." The `? / :` pair is a one-line *if / else*.

Node.js — JavaScript on the server

Normally JavaScript runs only inside a browser. **Node.js** is a separate program that lets you run JavaScript outside the browser — for example, on your laptop, or on a cloud server. When the README says "run `npm start`", what actually happens is: Node.js opens `server.js`, executes every line, and keeps the program alive forever to answer incoming web requests.

All the **backend** files in this project (`server.js`, `config.js`, the things under `db/`) are Node.js programs. They never touch the browser directly — they speak HTTP and SQL.

PostgreSQL — the database

A database is a program that stores structured data and lets other programs read and write it.

PostgreSQL (often shortened to "Postgres") is the database this project uses. It speaks **SQL** (*Structured Query Language*) — a small declarative language for asking questions like:

```
SELECT id, description, amount  
FROM entries  
WHERE user_id = $1 AND month = $2  
ORDER BY created_at DESC;
```

SQL

Read that as: "from the table called `entries`, give me the columns `id`, `description` and `amount`, for the rows where `user_id` matches the first parameter and `month` matches the second, sorted newest-first." The `$1` and `$2` are placeholders we fill in safely — never by glueing strings together (more on that under SQL injection in Part V).

npm — the package manager

Modern projects rarely write everything from scratch. They borrow code from thousands of small libraries (called **packages**). **npm** (Node Package Manager) is the program that downloads those packages into a `node_modules/` folder and keeps a record of which versions you are using in two files:

- `package.json` — the list of packages you *want* (and other metadata, like the project version).
- `package-lock.json` — the exact versions npm actually installed last time, down to dependencies of dependencies. This lock file is what makes "install once, get identical code on every machine" possible.

Open `package.json` to see this project depends on packages like `express` (a web framework), `pg` (PostgreSQL driver), `bcryptjs` (password hashing), `helmet` (security headers), `pdf-parse` (read PDF text), `pdfkit` (write PDFs), `@anthropic-ai/sdk`, `@google/genai`, `openai`, etc. We will meet each of those in context.

The big picture, in one paragraph

How the pieces talk

Your browser loads `index.html` (HTML + CSS + a couple of `<script>` tags). Inside those scripts, `js/app.js` runs and immediately calls `fetch("/api/user", ...)`. That request travels over the internet to `server.js`, which checks your session, asks PostgreSQL "who is this user?", and replies with JSON. `js/app.js` receives the JSON, paints the dashboard, and from then on every click that needs server help repeats the loop. Charts, AI chat, PDF upload, settings changes — all of them are little request/response cycles between the same two programs.

Part II — Map of the project

Every folder, every file, at a glance.

Here is what you will find at the root of the repository. We will return to each file in detail in the parts that follow; this section is the bird's-eye view.

Top-level layout

```
asset-management/
% % % package.json          - list of npm dependencies + version (3.0.2)
% % % package-lock.json     - exact versions of every installed package
% % % config.js             - loads environment variables, validates secrets
% % % server.js             - THE backend: 6,000+ lines, ~64 endpoints
% % % index.html            - main dashboard page (140 KB, 3,475 lines)
% % % login.html            - login page
% % % register.html         - sign-up page
% % % forgot-password.html  - password reset page
% % % js/
%   % % % app.js            - dashboard logic (5,600+ lines)
%   % % % chat.js           - AI chat widget
%   % % % i18n.js           - translations (English + Portuguese)
%   % % % csrf.js           - CSRF token helper (security)
%   % % % login.js          - login form handler
%   % % % register.js       - registration + PayPal
%   % % % forgot-password.js - password reset flow
% % % db/
%   % % % pool.js           - PostgreSQL connection pool
%   % % % queries.js        - every SQL query in the app
%   % % % schema.sql        - table definitions
%   % % % migrate-*.sql     - incremental schema migrations
%   % % % migrate-json-to-pg.js - one-shot upgrade from old JSON storage
%   % % % MIGRATION_RUNBOOK.md - operator notes
% % % ios/                  - Capacitor wrapper for the iOS app
% % % www/                  - copy of web assets used by the iOS build
% % % ssl/                  - local TLS certificates (gitignored)
% % % .env.example          - sample environment variables
% % % backup.sh             - database backup helper
% % % deploy.sh             - production deploy helper
% % % rotate-key.sh         - wrapper around the encryption-key rotator
% % % rotate-encryption-key.js - re-encrypts every stored secret
```

Two halves: client vs server

Every file in the project belongs to one of two universes:

- **Client-side** (runs in the browser): the four HTML files and the seven files under `js/`. The browser downloads them, executes them, and they only know about each other and the DOM.
- **Server-side** (runs in Node.js): `config.js`, `server.js`, and everything under `db/`. The browser never sees these files — and a security check explicitly blocks anyone trying to download them.

When a piece of code in the client needs information that only the server has (your entries, AI replies, etc.), it calls an **HTTP endpoint** — a URL like `/api/entries` — and gets JSON back. We will trace these calls end-to-end in Part VI.

What is not here

A few things are conspicuously missing for a project this size, and that is intentional:

- **No build step.** No webpack, no Vite, no TypeScript compiler. The browser receives the JS files exactly as they live on disk. Editing `js/app.js` and refreshing is the entire development loop.
- **No CSS files.** Every stylesheet is inlined inside its HTML file in a `<style>` block.
- **No test framework.** There are no unit tests; you sanity-check with `node --check server.js`.
- **No frontend framework.** No React, no Vue, no Svelte. Everything is "vanilla" JavaScript using the browser's built-in DOM APIs (`document.getElementById`, `addEventListener`, etc.).

Part III — The front end, page by page

What the user sees, and the JavaScript that drives it.

We start with the front end because it is the universe the user actually touches. Every HTML file follows the same general shape: head (metadata + styles + bootstrapping scripts), body (visible content), bottom-of-page `<script>` tags that load JS in order. Let us look at each one.

login.html — the door

A typical session begins here. The page contains a small form (username + password), a hidden block for the two-factor-authentication step, and links to "Register" and "Forgot password".

Notable details:

- At the very top of `<head>` an inline script reads `localStorage.getItem("appTheme")` and `localStorage.getItem("appTypography")` and applies them as attributes on `<html>` **before** the stylesheet finishes loading. Without that, dark-mode users would see a flash of light-mode colours on every navigation.
- A `<link>` tag loads Google Fonts (Instrument Serif, Geist, Inter, JetBrains Mono) over the network so the page can use those typefaces regardless of which fonts are installed on the user's computer.
- Below the form, three `<script>` tags load `js/i18n.js`, `js/csrf.js` and `js/login.js` — *in that order*, because `login.js` calls functions defined in the other two.

js/login.js — the form handler

All 116 lines are wrapped in `(function() { ... })()`, an old-school technique called an **Immediately Invoked Function Expression (IIFE)**. It exists to keep the variables inside private — they do not leak into the global scope. Inside, the file does three things:

- On `submit`, it sends the username + password to `POST /api/login` using `csrfFetch` (which automatically attaches the CSRF token).
- If the server replies `{ requires2FA: true, tempToken: "..." }`, it hides the password form and reveals the six-digit code input.
- On a successful login it redirects the browser to `/index.html`.

Notice how the file uses `t("login.errorDefault")` for every user-visible string. That is the `i18n` helper from `i18n.js` — we will dissect it shortly. Hard-coding text would mean re-translating it in every file.

register.html + js/register.js — sign-up + payments

`register.html` mirrors the login layout but adds an email field, password confirmation, an *invite code* field, and an optional **PayPal section** for buying an invite code directly.

`js/register.js` (208 lines) has two distinct halves:

- **PayPal flow.** It calls `/api/paypal/config` to ask if PayPal is enabled. If so, it injects the PayPal JS SDK at runtime, renders the credit-card button, and wires up `createOrder` / `onApprove` callbacks. When the user pays, the server issues a fresh invite code and the JS auto-fills it into the form.
- **Registration flow.** Client-side validation (email format, password length, allowed characters in usernames), then `POST /api/register` with the invite code attached, then a redirect to login on success.

Why client-side validation is not enough

Notice that the server also revalidates everything in `/api/register`. Treat the client as a UX nicety — it gives users instant feedback — but never as a security boundary. Anything sent from the browser can be forged, so the server must check the same rules itself.

forgot-password.html + js/forgot-password.js

A two-step form: enter your username (step 1), then enter the eight-character reset code and a new password (step 2). The JS file is only 117 lines because, again, all the real work happens on the server (`/api/forgot-password` and `/api/reset-password`).

One detail to highlight: regardless of whether the username exists, step 1 always advances to step 2. That is deliberate — it prevents an attacker from probing "does this username exist?" by watching the response.

index.html — the dashboard

This is the big one — 3,475 lines, ~140 KB. It defines the *entire* logged-in experience: the sidebar, the top bar with KPI cards, the chart panels, the entry list, the manage-categories modal, the budgets modal, the settings modal, the admin panel modal, the bulk-PDF-upload modal, the chat widget, and every CSS variable that controls how all of this looks.

The shape of the file

```

<!DOCTYPE html>          <- tells the browser this is modern HTML
<html lang="en">
<head>
  <meta ...>              <- character set, viewport
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <script>(theme bootstrap IIFE)</script>
  <script src="https://cdn.jsdelivr.net/.../chart.js"></script>
  <script src="https://cdn.jsdelivr.net/.../chartjs-plugin-annotation"></script>
  <style>
    /* ~2,500 lines of CSS – palettes, layout, components */
  </style>
</head>
<body>
  <aside class="sidebar"> ...navigation... </aside>
  <main>
    <header> ...top bar with KPIs... </header>
    <section class="charts"> ...4 chart canvases... </section>
    <section class="entries"> ...table + pagination... </section>
  </main>
  <!-- modals (initially hidden) -->
  <div class="modal" id="settingsModal"> ... </div>
  <div class="modal" id="manageCategoriesModal"> ... </div>
  <div class="modal" id="budgetsModal"> ... </div>
  <div class="modal" id="adminModal"> ... </div>
  <!-- chat widget -->
  <button id="chatFab">ðŸŒˆ</button>
  <div id="chatWindow"> ... </div>

  <script src="/js/i18n.js"></script>
  <script src="/js/csrf.js"></script>
  <script src="/js/app.js"></script>
  <script src="/js/chat.js"></script>
</body>
</html>

```

Where the CSS lives

Open the `<style>` block at the top and you will see a carefully organised cascade. The first block defines colour palette variables on `:root` (the document root):

```

:root {
  --bg: #F3ECE0;          /* warm sand */
  --card: #FBF6EC;        /* light cream – used for panels */
  --ink: #26201A;         /* near-black for body text */
  --primary: #B8593A;     /* terracotta – buttons, accents */
  --positive: #6B8248;    /* olive green – income */
  --negative: #B8593A;    /* terracotta – expenses */
  /* ...font families, shadows, radii... */
}

```

Further down, `html[data-theme="dark"] { ... }` and `html[data-theme="light"] { ... }` blocks redefine the same variables with different hex values. Switching themes just means setting one attribute on `<html>` — the cascade does the rest.

Typography presets work the same way: `html[data-typography="modern"]` swaps `--font-display` from a serif to a sans, and every heading that says `font-family: var(--font-display)` follows along.

What is `data-i18n="..."`?

Throughout the HTML you will spot attributes like `data-i18n="settings.title"` on otherwise-empty elements. These are markers used by `i18n.js`: after the DOM finishes loading, `applyTranslations()` walks every element with `data-i18n`, reads the key, and replaces the element's text content with the right string for the active language.

That is why almost no English text is hardcoded into the HTML — it is all looked up at runtime.

`js/app.js` — the dashboard's brain (5,610 lines)

This file is the single longest piece of code in the project after `server.js`. It is responsible for almost everything that happens in `index.html` after the page paints. We will not list all 114 functions — instead, we will group them into the seven jobs the file performs.

1. State (lines 1–110)

The top of the file declares the global variables that other functions read and write. There is no React-style reactive system — we manually call "re-render" functions when state changes.

```
let userCategories = []; // [{slug, label, color, ...}, ...]
let _userCategoriesBySlug = new Map(); // fast lookup
let entries = []; // every entry fetched from the server
let currentFilteredEntries = []; // what the table actually shows
let currentUser = null; // {id, username, role, partnerId, ...}

// chart instances – null until first render
let monthlyBalanceChart = null;
let incomeVsExpenseChart = null;
let categoryChart = null;
let categoryStackedChart = null;
```

JS

When data changes (say, the user adds an entry), we call functions like `updateCharts()`, `displayEntries()`, `updateHeroKpis()`, and they each redraw their piece of the screen using the new state.

2. Categories (issue #70)

Categories are per-user. The server seeds the same 17 defaults for every new account (food, transport, etc.) but the user can add, rename, recolour, or delete them. The frontend keeps a copy in `userCategories` and exposes three helper functions that everything else uses:

- `categoryColor(slug)` — hex colour used by charts and chips.
- `categoryLabel(slug)` — human-readable name. Defaults go through `t("cat.food")` for translation; custom categories use their stored label verbatim.
- `categorySlugList()` — the slugs in order, used to drive `<select>` dropdowns.

3. Charts (`Chart.js`)

The dashboard shows four charts, all driven by the third-party library **Chart.js** (loaded from `cdn.jsdelivr.net` in `index.html`). The file declares one variable per chart instance and rebuilds them via `initializeCharts()` and `updateCharts()`.

- **Monthly balance** — a line chart of running net (income minus expenses) over time, with an annotation marker for the current month.
- **Income vs. expense** — a paired bar chart per month.
- **Category chart** — either a horizontal bar chart or a doughnut, switchable via `setCategoryChartType()`.

- **Stacked category bar** — for the Reports modal: a stacked bar showing every category's contribution per month.

Why Chart.js?

Drawing charts by hand on an HTML `<canvas>` is possible but tedious. Chart.js wraps the math (axes, legends, animations, hit-testing for tooltips) behind a simple JSON config. Each chart instance owns one canvas element; calling `.destroy()` on it cleans up before re-creating with new theme colours (`reapplyChartTheme()` does this).

4. Filtering, sorting, pagination

Every time the user types in the search box, picks a quick range (This month, Last 3 months, etc.), or clicks a category chip, three things happen:

- `readFilterStateFromInputs()` collects the current filter values into a plain object.
- `saveFilterState()` persists that object to `localStorage` under a per-user key — so refreshing the page does not lose your filters.
- `filterEntries(opts)` produces a new array of entries, which then flows into `sortEntries()`, `displayEntries()`, `updateCharts()`, `updateHeroKpis()`, and `updateSummary()`.

Pagination is local (no server round-trip): the table renders 50 rows per page and `renderEntriesPagination()` draws the page-number buttons.

5. Loading-state helpers

Async operations (anything that talks to the server) take time. The helpers `setViewLoading`, `setHeroLoading`, `setChartsLoading`, `setEntriesLoading` and `setSingleChartLoading` toggle CSS skeleton classes plus `aria-busy="true"` on the right region so the page never looks frozen during a fetch.

For modal buttons, `setButtonLoading(btn, true)` adds a spinner pseudo-element and disables the button until the request resolves.

6. CRUD flows

CRUD stands for *Create, Read, Update, Delete*. The dashboard supports all four for entries, categories, and budgets. They all follow the same five-step recipe:

```

async function addEntry(payload) {
  setButtonLoading(submitBtn, true);
  try {
    const res = await csrfFetch('/api/entries', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload)
    });
    if (!res.ok) throw new Error(await res.text());
    const created = await res.json();
    entries.push(created);           // update local state
    await refreshDashboard();        // redraw everything
  } catch (err) {
    showToast(t('entries.errorGeneric'));
  } finally {
    setButtonLoading(submitBtn, false);
  }
}

```

Every mutation uses `csrfFetch` (not `raw fetch`) so the CSRF header is attached automatically. Every error message comes through `t(...)` so it is translated.

7. Modals & the mobile sidebar

All modals share the same skeleton: a full-screen overlay div that is hidden by default and revealed by setting a class. The "Manage categories", "Budgets", "Settings", "Admin panel", and "Reports" modals all open via small wrapper functions (e.g. `openManageCategoriesModal()`), trap focus inside the modal for keyboard users, and close on Esc.

On mobile, the sidebar slides in over the content; the focus-management code makes sure tab key navigation stays inside the drawer until it is closed.

js/chat.js — the floating AI advisor (675 lines)

A self-contained widget. The HTML for it lives in `index.html` (`#chatFab` is the floating button, `#chatWindow` is the conversation panel). This file owns the open/close behaviour, the message history, and the chat fetch.

The flow is:

- User clicks `#chatFab` -> `openChat()` reveals the panel and pushes a welcome message.
- User types and presses Enter -> `sendMessage()` appends the user's text to the local `chatMessages` array, renders it, then `POST /api/ai/chat` with the full history.
- Server may reply with either plain text (we render it as Markdown via `parseMarkdown()`) or with `pendingEdits` / `pendingDeletes` arrays — in which case we render an interactive **Confirm / Cancel** card per item.
- Clicking *Confirm* on an edit card calls `POST /api/ai/confirm-edit`; clicking *Cancel* calls `/api/ai/cancel-edit`. Deletes work the same way against `/api/ai/confirm-delete` / `/cancel-delete`.

The local `parseMarkdown()` function is interesting: it has to be defensive because the AI controls the text. It protects fenced code blocks, escapes HTML, handles tables, and explicitly avoids `eval()` or `innerHTML` on user content unless it has been through the escape step.

js/i18n.js — translations (1,321 lines)

The file is mostly one giant object literal:

```
const translations = {
  en: {
    'common.appName': 'Asset Manager',
    'common.logout': 'Logout',
    'cat.food': 'Food',
    /* ~700 keys */
  },
  pt: {
    'common.appName': 'Gestor de Ativos',
    'common.logout': 'Sair',
    'cat.food': 'Alimentação',
    /* same keys, Portuguese */
  }
};
```

The four public functions are tiny:

```

function getLang() {
  return localStorage.getItem('app-lang') || 'en';
}
function setLang(lang) {
  localStorage.setItem('app-lang', lang);
  location.reload();
}
function t(key, replacements) {
  const lang = getLang();
  let str = translations[lang]?.[key] || translations.en[key] || key;
  if (replacements) {
    Object.keys(replacements).forEach(p =>
      str = str.replace(new RegExp('\\{' + p + '\\}', 'g'), replacements[p])
    );
  }
  return str;
}
function applyTranslations() {
  document.querySelectorAll('[data-i18n]').forEach(el => {
    el.textContent = t(el.getAttribute('data-i18n'));
  });
  /* ...and the same for placeholders, titles, aria-labels... */
}

```

`applyTranslations()` is called automatically once at `DOMContentLoaded` and again whenever the user toggles the language. Because every translatable string in the HTML carries a `data-i18n="..."` attribute, the file never has to know which page it is on — it just walks the DOM.

js/csrf.js — the security gate for write requests (42 lines)

This tiny file is loaded by every page. It exposes two functions:

- `getCsrftoken()` — fetches a per-session CSRF token from `/api/csrf-token` once, caches it for the lifetime of the page, and returns the cached value on every subsequent call.
- `csrfFetch(url, options)` — a drop-in replacement for `fetch()`. For any non-GET/HEAD/OPTIONS request it transparently attaches the header `x-csrf-token: <token>`. The server compares this header against the token stored in your session cookie; if they do not match, the request is rejected.

You will never see a raw `fetch(...)` call in the rest of the front end for state-changing operations — always `csrfFetch(...)`. That is by convention, not enforcement; consider it a strict house rule.

Part IV — The back end, file by file

Where the security, the encryption, and the AI live.

Everything in this part runs on the server — never in the browser. If you ran `npm start` on your laptop right now, this is the code that would come to life.

config.js — the gatekeeper for secrets (73 lines)

config.js is the very first file the server runs. Its job is simple but critical: load environment variables, validate the ones the app cannot live without, and refuse to start otherwise.

```
require('dotenv').config();           // .env -> process.env (dev only) JS

const ENCRYPTION_KEY = process.env.ENCRYPTION_KEY;
const SESSION_SECRET = process.env.SESSION_SECRET;

if (!ENCRYPTION_KEY) { console.error('FATAL: ENCRYPTION_KEY is not set. ');
process.exit(1); }
if (!/^[0-9a-fA-F]{64}$/.test(ENCRYPTION_KEY)) { /* must be 64 hex chars */
process.exit(1); }
if (!SESSION_SECRET) { process.exit(1); }
if (SESSION_SECRET.length < 32) { process.exit(1); }

module.exports = {
  encryptionKey: Buffer.from(ENCRYPTION_KEY, 'hex'), // 32 raw bytes
  sessionSecret: SESSION_SECRET,
  adminUsername: process.env.ADMIN_USERNAME || 'admin',
  /* ...PayPal, SMTP, AI keys... */
};
```

Fail fast

The "process.exit(1)" pattern is deliberate. If ENCRYPTION_KEY were missing and the server silently picked a random value, every existing user would suddenly have unreadable data. Better to refuse to start than to corrupt data in transit.

db/pool.js — the database connection pool (48 lines)

Every time the server needs to talk to PostgreSQL, it borrows a connection from a **pool** — a small set of already-open TCP connections that get reused. Opening a fresh TCP + auth handshake per query would be ten times slower; the pool avoids that.

```
const { Pool } = require('pg');

const pool = new Pool({
  host:      process.env.PGHOST      || 'localhost',
  port:      parseInt(process.env.PGPORT || '5432', 10),
  database:  process.env.PGDATABASE || 'asset_management',
  user:      process.env.PGUSER,
  password:  process.env.PGPASSWORD,
  max:       10,                    // max concurrent connections
  idleTimeoutMillis: 30000,        // close idle ones after 30s
  connectionTimeoutMillis: 5000    // give up if connect takes >5s
});

module.exports = { pool, testConnection };
```

JS

Exporting a single shared pool means every query in `db/queries.js` borrows from the same set of connections. The `testConnection()` helper is called once at startup so the server crashes immediately if it cannot reach Postgres.

db/schema.sql — the table definitions (132 lines)

PostgreSQL is a **relational** database, which means data lives in tables with named columns and explicit relationships. `schema.sql` is the master script you run once on a fresh install (`psql -U asset_app -d asset_management -f db/schema.sql`) to create every table.

There are seven tables:

- **users** — one row per account: username, bcrypt password hash, role, encrypted email + API keys + TOTP secret, partner_id, etc.
- **entries** — one row per income/expense item: user_id, month (YYYY-MM), type, amount, description, tags, is_couple_expense.
- **user_categories** — per-user tag list (defaults + customs).
- **user_budgets** — per-user monthly target per category, plus a "_overall" budget.
- **invite_codes** — required for registration; admins create them or buyers acquire them via PayPal.
- **paypal_orders** — record of every PayPal transaction.
- **session** — managed automatically by the `connect-pg-simple` library to persist Express sessions across restarts.

Notable details inside the file:

```
CREATE TABLE IF NOT EXISTS entries (
  id          BIGSERIAL PRIMARY KEY,
  user_id     BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  month       TEXT NOT NULL CHECK (month ~ '^\d{4}-(0[1-9]|1[0-2])$'),
  type        TEXT NOT NULL CHECK (type IN ('income', 'expense')),
  amount      NUMERIC(15,2) NOT NULL CHECK (amount > 0),
  description TEXT NOT NULL,
  tags        TEXT[] NOT NULL DEFAULT '{}',
  is_couple_expense BOOLEAN NOT NULL DEFAULT FALSE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

SQL

- `BIGSERIAL PRIMARY KEY` auto-numbers each row (1, 2, 3, ...).
- `REFERENCES users(id) ON DELETE CASCADE` is a **foreign key**: when a user is deleted, all of their entries disappear automatically — the database guarantees that.

- CHECK constraints validate at the database level — even a hand-written SQL query cannot insert a row with a malformed month or a negative amount.
- NUMERIC(15, 2) is the right type for money: it never loses precision the way float does.
- TEXT[] is a PostgreSQL array column — tags can hold multiple slugs without a separate join table.

Below the table definitions, indexes are declared:

```
CREATE INDEX IF NOT EXISTS idx_entries_user_id_month
  ON entries(user_id, month);
CREATE UNIQUE INDEX IF NOT EXISTS idx_users_username_lower
  ON users(LOWER(username));
```

SQL

Indexes

Without an index, Postgres reads every row of "entries" to find yours. With `idx_entries_user_id_month`, it jumps straight to your rows in microseconds. The unique index on `LOWER(username)` enforces case-insensitive uniqueness — "ALICE" and "alice" cannot both register.

db/queries.js — every SQL query in the app (1,348 lines)

Every other file calls into `queries.js` rather than writing SQL inline. This rule has two reasons: parameterised queries are easier to keep safe from SQL injection, and concentrating SQL in one file makes audits and schema changes manageable.

Shape conversion

PostgreSQL columns use snake_case (e.g. `password_hash`) but JavaScript objects in this project use camelCase (`passwordHash`). The file opens with two helpers that translate raw DB rows into the shapes the rest of the code expects:

```
function dbRowToUser(row) {
  return {
    id:          Number(row.id),
    username:    row.username,
    passwordHash: row.password_hash,
    role:        row.role,
    email:       parseJsonField(row.email),           // {iv, encryptedData}
    geminiApiKey: parseJsonField(row.gemini_api_key),
    /* ...etc... */
  };
}
```

JS

Encrypted columns are stored as JSON text like `{"iv":"...", "encryptedData":"..."}`. `parseJsonField` runs `JSON.parse` on them; if parsing fails, it logs only the error name (never the input) to avoid accidentally leaking the secret into logs — a real CodeQL finding noted in the comments.

Parameterised queries

Every query uses positional placeholders. We never concatenate user input into SQL strings.

```

async function findUserByUsername(username) {
  const { rows } = await pool.query(
    'SELECT * FROM users WHERE LOWER(username) = LOWER($1)',
    [username] // <- `username` flows in as a parameter, not as SQL
  );
  return dbRowToUser(rows[0]);
}

```

JS

SQL injection

If we wrote `"SELECT ... WHERE username = " + username + ""` instead, a user named "admin" OR `1=1 --` would log in as somebody else. Parameter binding tells PostgreSQL "treat this as a value, not as code"; it is the single most important defence against SQL injection.

Selected functions worth knowing

- `getEntriesByUser(userId, month)` — base query for the "Individual" view.
- `getCoupleEntries(userId, partnerId, month)` — Individual + partner-only couple entries, for the "Combined" view.
- `getMyShareEntries(userId, partnerId, month)` — half of couple entries + 100% of personal, for the "My Share" view.
- `findDuplicateEntry(...)` and `findBulkDuplicateEntries(...)` — used by the bulk PDF upload to warn when an extracted line matches one already in the database.
- `createInviteCodeIfNotExists(code, createdBy)` — atomic insert that returns `null` on conflict, so the caller can retry with a new random code.
- `addUserCategoryAtomicWithCap(...)` — wraps the insert in `pg_advisory_xact_lock(userId)` so two simultaneous requests cannot push the per-user category count above 100.
- `upsertUserBudget(userId, categorySlug, amount)` — uses `INSERT ... ON CONFLICT (... COALESCE(category_slug, '')) DO UPDATE` to support the special `_overall` budget (where `category_slug` is `NULL`).

db/migrate-*.sql — incremental schema migrations

When the application gains a new feature that needs new tables or columns, we never edit `schema.sql` and ask operators to "drop and re-create everything". Instead we ship a small migration file:

- `migrate-add-claude-oauth-token.sql` — adds the `claude_oauth_token` column to `users`.
- `migrate-add-github-copilot-token.sql` — adds the `github_copilot_token` column.
- `migrate-add-user-categories.sql` — creates `user_categories`.
- `migrate-add-user-budgets.sql` — creates `user_budgets` + the unique index.
- `migrate-add-indexes.sql` — adds performance indexes.

Each one is wrapped in `BEGIN; ... COMMIT;` and uses `IF NOT EXISTS` so running it twice is harmless. `MIGRATION_RUNBOOK.md` is the operator-facing checklist that explains, for each release, which migrations to run and in what order.

db/migrate-json-to-pg.js — the one-shot upgrade (259 lines)

Before version 2 this project stored everything in two AES-encrypted JSON files (`data/users.json`, `data/entries.json`). This script is what existing deployments ran *once* to move their data into PostgreSQL.

It is interesting for what it teaches:

- It reads the old JSON, decrypts it with the same `ENCRYPTION_KEY` the server uses, and then re-inserts everything inside **a single PostgreSQL transaction** (`BEGIN; ... COMMIT;`). If anything fails halfway, the transaction rolls back and the database is untouched.
- Every insert uses `ON CONFLICT DO NOTHING` so re-running the script after a partial run is safe.
- It never modifies the JSON files — operators can keep them as a backup until they are confident.

rotate-encryption-key.js — re-encrypt everything (206 lines)

If the `ENCRYPTION_KEY` is ever suspected to be compromised, you need to re-encrypt every stored secret with a fresh key. This script:

- Loads the *current* key via `config.js`.
- Generates a new 32-byte key with `crypto.randomBytes(32)`.
- For every encrypted column (`email`, `gemini_api_key`, `openai_api_key`, `anthropic_api_key`, `claude_oauth_token`, `github_copilot_token`, `totp_secret`): decrypt with the old key, re-encrypt with the new key, and `UPDATE` the row — all inside one transaction.
- Prints the new key on success. The operator must then update the systemd unit and restart the server.

The accompanying shell script `rotate-key.sh` automates the systemd dance: stop the service, run the Node script, write the new key into the unit override, restart.

server.js — the monolith (6,083 lines)

Everything else feeds into this file. It is intentionally a monolith: a single Express application that defines all middleware, all routes, all AI integration glue, and all helper functions. We will walk through it in nine layers, in the same order the code itself defines them.

1. Imports and version

```
const config      = require('./config');
const express     = require('express');
const session     = require('express-session');
const PgSession   = require('connect-pg-simple')(session);
const bcrypt      = require('bcryptjs');
const helmet      = require('helmet');
const crypto      = require('crypto');
const db          = require('./db/queries');
const { pool: dbPool, testConnection: testDbConnection } = require('./db/pool');
const multer      = require('multer');      // multipart/form-data (file uploads)
const rateLimit   = require('express-rate-limit');
const pdfParse    = require('pdf-parse');   // extract text from PDFs
const PDFDocument = require('pdfkit');     // generate PDFs (Reports export)
const { GoogleGenAI, Type } = require('@google/genai');
const OpenAI      = require('openai');
const Anthropic   = require('@anthropic-ai/sdk');
const nodemailer  = require('nodemailer');
const otplib      = require('otplib');     // TOTP codes (2FA)
const QRCode      = require('qrcode');     // QR for 2FA enrolment
```

JS

`APP_VERSION` is read from `package.json` at boot, so bumping the version is a one-line change.

2. Encryption helpers

AES-256-CBC is a symmetric cipher: the same 32-byte key both encrypts and decrypts. We use it for every stored secret.

```
const ALGORITHM = 'aes-256-cbc';

function encryptString(value) {
  const iv = crypto.randomBytes(16); // fresh IV per value
  const cipher = crypto.createCipheriv(ALGORITHM, ENCRYPTION_KEY, iv);
  let encrypted = cipher.update(value, 'utf8', 'hex');
  encrypted += cipher.final('hex');
  return { iv: iv.toString('hex'), encryptedData: encrypted };
}

function decryptString(encryptedData, iv) {
  const decipher = crypto.createDecipheriv(ALGORITHM, ENCRYPTION_KEY,
    Buffer.from(iv, 'hex'));
  let decrypted = decipher.update(encryptedData, 'hex', 'utf8');
  decrypted += decipher.final('utf8');
  return decrypted;
}
```

JS

What is an IV?

IV stands for Initialisation Vector — a random 16-byte value mixed into the cipher state. Using a fresh IV per encryption means that encrypting the same plaintext twice produces two completely different ciphertexts. Without that, an attacker could spot "user A and user B have the same Gemini key" just from the database.

3. Anthropic & GitHub Copilot auth resolution

Anthropic accepts two kinds of credential: a regular API key (sk-ant-api03-...) sent via x-api-key, or a Claude Code OAuth token (sk-ant-oat01-...) sent via Authorization: Bearer ... plus a special beta header. `resolveAnthropicAuth(user)` picks the right one in a defined order (user OAuth -> user API key -> env OAuth -> env API key) and `createAnthropicClient(...)` wires up the right headers.

GitHub Copilot is even fiddlier. The exchange flow is:

- Take the user's long-lived GitHub OAuth token (gho_... / github_pat_...).
- Call GET https://api.github.com/copilot_internal/v2/token with that token to get a short-lived (~30 min) **session token**.
- Parse proxy-ep=... out of the session token to find the per-account API endpoint (api.individual.githubcopilot.com, etc.).
- Cache the session token + base URL per user, refreshing 60 seconds before expiry.
- Every chat completion call uses an OpenAI SDK client pointed at that base URL, with Editor-Version, User-Agent, X-Initiator, Openai-Intent headers that make it look like the official VS Code Copilot extension. Without those headers Copilot returns misleading 401s.

4. Brute-force protection & reset codes

Two in-memory Map structures (`failedLoginAttempts`, `resetAttempts`) record failures per username and per (username, IP). Thresholds escalate (5 attempts -> 15 minute lockout, 10 attempts -> 1 hour). Stale records are pruned every 30 minutes by a `setInterval` cleanup loop.

Password-reset codes are 8 random characters from A-Z0-9, generated with **rejection sampling** so every character is equiprobable (the comment in the source explains why naive `byte % 36` is biased — it overrepresents the first four characters by ~14%).

5. Security middleware (helmet, dotfile blocker, CSP)

```
app.use(helmet({JS
  crossOriginOpenerPolicy: { policy: 'same-origin-allow-popups' },
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ['self'],
      scriptSrc: ['self', "'unsafe-inline'", "https://cdn.jsdelivr.net",
        "https://cloud.umami.is", "https://*.paypal.com", ...],
      connectSrc: ['self', "https://cloud.umami.is", "https://*.paypal.com",
...],
      frameSrc: ["https://*.paypal.com"],
      objectSrc: ['none'],
      /* etc. */
    }
  }
}));
```

Helmet sets a bundle of HTTP security headers — Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, etc. The CSP above whitelists exactly the third-party domains we trust (CDN for Chart.js, PayPal, Umami analytics).

A small custom middleware below it blocks any URL that touches dotfiles (`/.env`, `/.git`) or sensitive server-side paths (`/server.js`, `/db`, `/node_modules`). This is belt-and-braces: even if static-file middleware were misconfigured, requests for those files would still 404.

6. HTML caching + cache-busting

Each HTML file is read once at startup, modified to inject the Umami analytics script (if configured) and to append `?v=<BUILD_ID>` to every `<script src>` and `<link href>`, then cached in memory. Subsequent requests serve the precomputed string straight from the cache:

```
const BUILD_ID = Date.now().toString(36); // unique per process startJS
function injectCacheBust(html) {
  return html
    .replace(/(<script\b[^\>]*\bsrc=")(\/[^\?]+\.(js)(")/g, ` $1$2?v=${BUILD_ID}$3`)
    .replace(/(<link\b[^\>]*\bhref=")(\/[^\?]+\.(css)(")/g, ` $1$2?v=${BUILD_ID}$3`);
}
```

The CDN browser cache happily reuses `/js/app.js?v=abc` for an hour. After deploying a new version, the build ID changes (`/js/app.js?v=xyz`), browsers see a different URL and re-fetch. That is the entire cache-bust mechanism.

7. Sessions and CSRF

```
app.use(session({JS
  store: new PgSession({ pool: dbPool, tableName: 'session',
    createTableIfMissing: true,
    pruneSessionInterval: 60 * 15 }),
  secret: config.sessionSecret,
  resave: false, saveUninitialized: false,
  cookie: { secure: process.env.NODE_ENV !== 'development',
    httpOnly: true, sameSite: 'lax',
    maxAge: 24 * 60 * 60 * 1000 } // 24h
}));
```

Sessions live in the `session` table in PostgreSQL — restarting the server does not log everybody out. The session ID travels in a signed cookie (`connect.sid`); the rest of the session data (`req.session.user`, etc.) is read from the database on each request.

Right after session middleware we get the CSRF flow. `/api/csrf-token` lazily creates a random per-session token, and a global middleware enforces that every non-GET request must carry that exact token in the `x-csrf-token` header. Compared with `crypto.timingSafeEqual`, the comparison is constant-time — no early-exit timing side-channels.

8. Rate limiting

`express-rate-limit` is configured separately for every sensitive endpoint:

- Login: 10/15min by IP.
- Register: 5/hour by IP.
- PDF upload: 20/hour by user id (falling back to IP).
- Report export: 10/15min by user.
- PayPal order create: 5/hour.
- TOTP verify: 10/15min.
- Chat: 30/15min by user.
- AI models listing: 10/minute.
- General `/api/*`: 100/15min for unauthenticated requests; signed-in users skip this limiter.

9. Authentication middleware

Every protected endpoint declares `requireAuth` (and sometimes also `requireAdmin`). `requireAuth` is short but interesting:

```
const requireAuth = async (req, res, next) => {  
  if (req.session?.user?.id) {  
    try {  
      let user = getCacheUser(req.session.user.id);  
      if (user === undefined) {  
        user = await db.findUserById(req.session.user.id);  
        if (user) setCacheUser(user);  
      }  
      if (user && user.isActive) { req.user = user; return next(); }  
      req.session.destroy();  
      return res.status(401).json({ message: 'Session invalid. Please log in  
again.' });  
    } catch (err) {  
      console.error('Auth middleware DB error:', err.message);  
      return res.status(503).json({ message: 'Service temporarily unavailable.  
...' });  
    }  
  }  
  res.status(401).json({ message: 'Unauthorized' });  
};
```

JS

A 5-second user cache (`userCache`) means the database is hit at most once every five seconds per user, even if the dashboard issues many simultaneous fetches. Mutating endpoints (update / delete / link couple) wrap the original `db.update*` functions to call `invalidateCacheUser` so stale data is never served.

server.js — the routes

There are ~64 routes. Listing every line of every one would not be useful; instead, here is the route table grouped by purpose. For each, the line number is included so you can jump to it.

Authentication routes

- POST /api/login (1212) — checks brute-force lockout, compares bcrypt, optionally issues a temp 2FA token.
- POST /api/login/verify-2fa (1936) — second leg if 2FA is enabled; validates TOTP or a backup code.
- POST /api/register (1269) — public sign-up, requires a valid invite code, runs strict format validation.
- POST /api/forgot-password (1350) — always returns success to prevent username enumeration; only sends an email if the username matches a user with a stored email.
- POST /api/reset-password (1385) — consumes the 8-char reset code (one-time use, 15-min expiry).
- POST /api/logout (2195) — destroys the session and clears the cookie.

User settings

- GET /api/user (1433) — returns the current user object (with secrets stripped).
- GET / PUT /api/user/email (2024, 2044) — read & change the encrypted email.
- POST / DELETE /api/user/gemini-key | openai-key | anthropic-key | claude-oauth-token | github-copilot-token — store / clear an encrypted per-provider credential.
- PUT /api/user/ai-provider (1701) — switch active provider.
- PUT /api/user/ai-model (1904) — switch active model (validated against provider).
- PUT /api/user/web-search-toggle (1723) — enable/disable Anthropic-native web_search inside the chat.
- GET /api/ai/models (1733) — list available models for the active provider. Cached by provider:keyHash for 5 minutes.
- GET / POST / VERIFY / DISABLE /api/user/2fa/* (2076–2151) — TOTP enrolment + backup-code generation.

Entries

- GET /api/entries?viewMode=...&month=... (2208) — supports individual, combined, myshare.
- POST /api/entries (2265) — create.
- POST /api/entries/check-duplicates (2326) — bulk duplicate detection before confirming a PDF upload.
- PUT /api/entries/:id (2402) — update.
- DELETE /api/entries/:id (2463) — delete.
- GET /api/reports/export?format=csv|pdf&...filters (2696) — streams a CSV or generates a PDF report.

Categories and budgets

- GET /api/categories (2990) — list, seeding the 17 defaults on first read.
- POST /api/categories (2995) — add (capped at 100/user via pg_advisory_xact_lock).
- PATCH /api/categories/:slug (3035) — rename / recolour / re-order.
- DELETE /api/categories/:slug (3066) — remove (existing entries keep the tag as an orphan).
- POST /api/categories/reset-defaults (3076) — restore the default set.
- GET /api/budgets (2831) — list, with the special _overall slug for the no-category target.
- PUT /api/budgets/:slug (2910) — upsert.

- DELETE /api/budgets/:slug (2951) — remove.

Admin & couples

- GET /api/admin/users (3102) — list users (admin only).
- POST /api/admin/users (3137) — create user.
- PUT /api/admin/users/:id (3177) — activate / deactivate / role.
- DELETE /api/admin/users/:id (3230) — delete.
- GET /api/admin/couples (3265) / POST /admin/couples/link | unlink — manage partner links.
- GET / POST / DELETE /api/admin/invite-codes (3447 / 3436 / 3474) — manage invite codes.

Payments

- GET /api/paypal/config (3324) — return client ID and price for the front end.
- POST /api/paypal/create-order (3333) — create an order on PayPal's side (CSRF-exempt — PayPal cannot carry our token).
- POST /api/paypal/capture-order/:orderId (3373) — finalise; on success, generate a fresh invite code and link it to the order.

AI features

- POST /api/ai/chat (4739) — the chat advisor. Loops through tool calls until the model is done (see Part V).
- POST /api/ai/confirm-edit | cancel-edit | confirm-delete | cancel-delete (5417 / 5459 / 5482 / 5534) — finalise the two-phase edit/delete proposals raised by the chat.
- POST /api/process-pdf (5573) — multipart upload of a PDF, parsed with pdf-parse, then handed to the active AI provider with a structured-output schema to produce entry candidates for the bulk-import preview.

Error handler and bootstrap

At the very bottom of `server.js` lives a global error handler that catches anything an `asyncHandler` did not handle, returns generic 500 JSON, and never leaks stack traces to clients. Below it, an `async IIFE` tests the database connection, runs the admin migration, calls `app.listen(PORT, '0.0.0.0', ...)`, and prints a verification line per configured integration (SMTP / PayPal / Gemini / OpenAI / Anthropic / Claude OAuth / Copilot OAuth).

Part V — Cross-cutting concepts

The big ideas that span many files.

Authentication, in full

Putting all the layers together, here is what happens from "I open the website" to "I see my dashboard":

- Browser requests / -> server serves `login.html` if you are not logged in (or `/index.html` if you visit it directly while logged out — but a JS-level check redirects you back).
- You submit the form -> `js/login.js` calls `csrfFetch("/api/login", { method: "POST", ... })`.
- `server.js` checks the brute-force counter, looks up the user, runs `bcrypt.compare(password, user.passwordHash)`. To prevent timing attacks against non-existent usernames, it compares against a precomputed `DUMMY_HASH` when the user does not exist — both code paths take the same time.
- If 2FA is enabled, a short-lived "temp token" is returned and the password is not yet enough; the front end shows the TOTP step and posts the code to `/api/login/verify-2fa`.
- On success, `req.session.user = { id, username, role, ... }` is set. Express serialises that into the `session` table, signs the session ID, and sets a `connect.sid` cookie.
- Future requests carry the cookie automatically (`credentials: "include"` on every fetch). `requireAuth` reads the session, loads the user (via the 5s cache), and lets the request through.

CSRF protection, in full

Cross-Site Request Forgery is the attack where a malicious site sneakily makes your browser issue a request to ours, using your existing cookie. We block it as follows:

- Every page calls `getCsrftoken()` (in `js/csrf.js`) once at startup, which fetches `/api/csrf-token` and caches the result. The server stores the token in `req.session.csrfToken`.
- For every non-GET request, `csrfFetch` attaches `x-csrf-token: <value>`. The server's CSRF middleware constant-time-compares the header against `req.session.csrfToken`. Mismatch -> 403.
- A malicious site cannot read the token (browser same-origin policy blocks reading other origins' fetch responses), and cannot guess it (it is 32 random bytes), and cannot set the header on a `<form>` submission (browsers refuse to add custom headers cross-origin without CORS preflight). So our endpoint sees the request, sees the missing/invalid token, and refuses.

The PayPal callbacks are the only exception. PayPal cannot carry our session token (it is on a different origin), so `/api/paypal/create-order` and `/api/paypal/capture-order/:orderId` are listed in a skip list. The `order_id` is the only thing the attacker would have to guess, and the order is bound to the IP / cookie that created it.

Encryption at rest

Eight pieces of user data are encrypted in the database with AES-256-CBC, keyed by `ENCRYPTION_KEY`:

- Email (so dumps of the users table cannot link accounts to identities).
- Gemini, OpenAI, and Anthropic API keys.
- Claude Code OAuth token, GitHub Copilot OAuth token.
- TOTP secret (used to generate the rolling 6-digit codes).

- Backup codes are stored as bcrypt hashes — not reversibly encrypted.

The server holds `ENCRYPTION_KEY` in memory only (loaded from `process.env`). If the database is exfiltrated alone, the attacker still cannot read any of those fields. If both the database and the key are lost, the `rotate-encryption-key.js` script gives operators an offline way to re-encrypt everything with a fresh key once the leak is contained.

Couples — how shared data flows

Two users can be "linked" by an admin. Once linked:

- Either partner can mark a new entry with `isCoupleExpense = true`. That entry still belongs to *one* user (the creator), but the partner can see it in their `Combined` view.
- The dashboard offers three view modes: `individual` (only your own entries), `combined` (your entries + partner's couple-flagged entries), and `myshare` (your personal + half of every couple entry).
- `db.ensurePartnerCategories(userId, partnerId, month)` runs on entry-list fetches to lazily import any partner-only category slug into your catalogue, so your filter chips render correctly the first time you switch to `Combined` view.
- AI chat tool responses include an `owner` field (me or partner) and an `editable` boolean. The chat is explicitly prevented from editing or deleting entries owned by the partner.

AI chat — the loop

When the user sends a chat message, the server runs an agentic loop that goes through possibly several rounds:

1. Build the system prompt + tool definitions for the active provider.
2. Prepend system role; append the user message to the in-flight message list.
3. Send the conversation to the provider (Gemini / OpenAI / Anthropic / Copilot).
4. If the model emits a tool call:
 - a. Look up the tool function (`getFinancialSummary`, `searchEntries`, ...)
 - b. Run it against the user's real data (parameterised SQL via `db/queries.js`)
 - c. Append `{ role: 'tool', content: <result> }` to the message list
 - d. Send the new list back to the provider — go back to step 3
5. When the model returns plain text (no tool call), send it to the client.
6. If `editEntry/deleteEntry` was proposed, also send a `pendingEdits / pendingDeletes` array so the chat widget can render `Confirm/Cancel` cards.

There are eight tools the model can call:

- `getFinancialSummary(startMonth, endMonth, coupleFilter?)`
- `getCategoryBreakdown(type?, startMonth?, endMonth?, coupleFilter?)`
- `getMonthlyTrends(startMonth?, endMonth?, coupleFilter?)`
- `getTopExpenses(limit?, category?, startMonth?, endMonth?, coupleFilter?)`
- `comparePeriods(period1Start, period1End, period2Start, period2End, coupleFilter?)`
- `searchEntries(keyword?, category?, type?, startMonth?, endMonth?, coupleFilter?, limit?)`
- `editEntry(entryId, ...new fields)` — two-phase: the AI proposes, the user confirms via UI.
- `deleteEntry(entryId)` — also two-phase. `undoLastEdit(entryId)` restores the previous snapshot.

All four providers share these tools but with provider-specific declaration formats (Gemini uses ``Type.OBJECT``, OpenAI/Copilot use the JSON-schema form, Anthropic uses its own `input_schema` format). The `chatToolDeclarations` and `openaiToolDeclarations` arrays at the top of

the AI section define both shapes.

Rate limiting and abuse

Beyond the per-endpoint limits already mentioned, three subtler defences:

- The login endpoint uses a precomputed DUMMY_HASH to prevent username enumeration via timing (the bcrypt.compare runs even if the user does not exist).
- The forgot-password endpoint always returns a success message regardless of whether the username matched, so an attacker cannot probe for valid usernames.
- Failed reset attempts are tracked per (username, IP) pair, capped at 5 per 15 minutes.

PDF processing pipeline

Bulk import is the most complex single feature. The flow:

- User picks a PDF in the browser and clicks Upload -> POST /api/process-pdf (multipart, max 10 MB).
- Server calls pdf-parse on the buffer to extract raw text.
- Server builds a per-user category list (defaults + customs + lazily imported partner couple-tag slugs).
- Server sends the text + the user's category list to the active AI provider with a strict JSON-schema response_format. Gemini and Anthropic also accept the PDF bytes directly via vision/file inputs.
- Provider returns a list of { month, type, amount, description, tag } candidates. The server normalises and returns them.
- Front end renders them in a preview table; the user can edit any field, delete rows, or accept.
- On confirm, the front end runs /api/entries/check-duplicates against the batch, optionally lets the user resolve duplicates, then issues a flurry of POST /api/entries calls.

Part VI — End-to-end walkthroughs

Trace a click from the button to the database and back.

Reading code one file at a time is hard. Reading it one request at a time is easier — every line in the chain has a purpose. Here are three walkthroughs at increasing complexity.

Walkthrough 1: "Add a new expense"

You click +, fill in the form, press Save.

1. Browser: app.js -> submit handler builds a payload
`{ month: '2026-05', type: 'expense', amount: 12.50, description: 'Coffee', tags: ['food'], isCoupleExpense: false }`
2. Browser -> server: `csrfFetch('/api/entries', POST, JSON body, x-csrf-token header)`
3. Server middleware chain:
 - helmet -> adds security headers
 - dotfile blocker -> not relevant for /api/
 - express.json() -> parses body into req.body
 - session -> reads connect.sid cookie, loads req.session from DB
 - CSRF middleware -> constant-time compares header vs session token
 - generalLimiter -> skipped because req.session.user exists
 - requireAuth -> loads req.user (cache hit if recent)
4. Route handler (server.js:2265):
 - Validates month regex, type " {income, expense}, amount > 0, description "d 500 chars, tags match SLUG_REGEX.
 - If isCoupleExpense=true, double-checks that the user actually has a mutually-linked active partner.
 - Calls `db.createEntry({ userId, month, type, amount, description, tags, isCoupleExpense })`.
5. `db/queries.js:592` – `INSERT INTO entries (...) VALUES ($1,$2,...) RETURNING *;`
`pool.query()` borrows a connection, sends the parameterised query, gets the inserted row back, `dbRowToEntry()` converts `snake_case` -> `camelCase`.
6. Server -> browser: 201 Created, JSON body of the new entry.
7. Browser: app.js pushes the new entry into the local 'entries' array, then calls `displayEntries()`, `updateCharts()`, `updateHeroKpis()` – the new row appears at the top of the table and the totals tick up.

Walkthrough 2: "Ask the AI: how much did I spend on food in March?"

Open chat, type the question, press Enter.

1. chat.js -> POST /api/ai/chat with body { message: '...', messages: [history] }
2. server.js:4739 – provider resolution: which AI vendor is this user on?
 - Resolves API key (per-user encrypted -> env fallback).
 - For Anthropic: resolveAnthropicAuth() returns either {authToken} or {apiKey}.
 - For Copilot: exchanges GitHub OAuth -> session token at request time.
3. Build system prompt + tools (chatToolDeclarations or openaiToolDeclarations).
4. Send conversation to the provider. The model usually replies with a tool call: getCategoryBreakdown({ type: 'expense', startMonth: '2025-03', endMonth: '2025-03' }).
5. Server runs the tool locally:
 - Fetches the user's entries via db.getEntriesByUser(userId, '2025-03') (or the couple variant, depending on view).
 - Groups by tag, sums amounts, returns { categories: [{slug, total, pct}, ...] }.
6. Server appends a 'tool' role message with the JSON result, sends back to the provider. (Anthropic uses 'tool_use' / 'tool_result' content blocks; OpenAI uses 'tool' role; Gemini uses functionCall / functionResponse parts.)
7. Provider sees the data and replies with natural-language text: "In March you spent R\$ 482.10 on food across 14 transactions..."
8. Server returns { content: '...', sources: [], pendingEdits: [] } to chat.js, which renders it via parseMarkdown().
9. The next message in the conversation reuses the same 'messages' array so the model has context for follow-up questions.

Walkthrough 3: "Set a \$300 monthly budget for transport"

1. User: opens Budgets modal -> app.js: openBudgetsModal()
2. Modal renders rows for each user category + an "overall" row.
3. User types 300 in the Transport row -> blur fires.
4. app.js: csrfFetch('/api/budgets/transport', {
 method: 'PUT', headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({ amount: 300 })
 });
5. Server (server.js:2910):
 - Validates slug matches SLUG_REGEX or is the special '_overall'.
 - Validates amount is a non-negative finite number.
 - Calls db.upsertUserBudget(userId, slug === '_overall' ? null : slug, 300).
6. db/queries.js:1254 – INSERT INTO user_budgets (user_id, category_slug, amount, period, currency)


```
VALUES ($1, $2, $3, 'monthly', 'USD')
ON CONFLICT (user_id, (COALESCE(category_slug, '')), period)
DO UPDATE SET amount = EXCLUDED.amount,
              updated_at = NOW()
RETURNING *;
```
7. The unique index uses COALESCE(category_slug, '') so a NULL category_slug (the overall budget) participates in the uniqueness check just like a normal slug. Otherwise two NULL rows could coexist.
8. Server returns the upserted row; the modal updates its UI.

Part VII — How to run, deploy, and maintain the app

Local development

```
# 1. Prerequisites BASH
#   Node.js "e 18.18 (because connect-pg-simple needs modern fetch/AbortController)
#   PostgreSQL "e 13

# 2. Configure secrets
cp .env.example .env
# edit .env: ENCRYPTION_KEY, SESSION_SECRET, ADMIN_PASSWORD_HASH, PG* vars

# 3. Generate the ENCRYPTION_KEY (32 random bytes -> 64 hex chars)
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

# 4. Generate the admin password hash
npm run hash-password -- MyStrongPassword123

# 5. Install dependencies
npm install

# 6. Create the database
createdb asset_management
psql -d asset_management -f db/schema.sql

# 7. Start
npm start # listens on http://localhost:3000
```

Production

In production the server is launched as a systemd unit, listens on port 443, and sits behind Nginx for TLS termination. The `deploy.sh` script is a thin wrapper around `git pull` + `npm ci` + `systemctl restart`.

Database backups are handled by `backup.sh`, which runs `pg_dump` nightly via cron. The `MIGRATION_RUNBOOK.md` lists which `migrate-*.sql` files must be applied per release; running them out of order is unsafe.

Updating the version

CLAUDE.md says it best: `APP_VERSION` is read from `package.json` at boot. To cut a new release, bump the version in `package.json`, commit, push, and the next restart reflects the change everywhere — login footers, chat headers, the "About" line in Settings. No constants to hunt for.

Rotating the encryption key

If you have any reason to suspect the key is leaked, do this immediately:

```
sudo systemctl stop asset-management BASH
node rotate-encryption-key.js # decrypts with old key, re-encrypts with a fresh one
sudo systemctl edit asset-management # update ENCRYPTION_KEY to the new value
sudo systemctl daemon-reload
sudo systemctl start asset-management
```

Reading the logs

Standard Node logging: every `console.log` / `console.error` goes to `journalctl` when running under `systemd`. The server prints a verification line per integration at startup, so you can spot at a glance whether SMTP, PayPal and each AI provider initialised correctly.

Part VIII — A glossary of every jargon term used

Skim this if a word ever stopped you mid-paragraph.

AES-256-CBC

A symmetric encryption algorithm. "AES" is the cipher, "256" is the key size in bits, "CBC" (Cipher Block Chaining) is the mode that lets you encrypt arbitrarily long inputs. The "256" part means brute-forcing the key would take longer than the heat death of the universe; the IV (Initialisation Vector) makes sure encrypting the same plaintext twice produces different ciphertexts.

Async / await

JavaScript syntax for working with operations that complete in the future (network calls, database queries, file reads). "async function foo()" returns a Promise; "await x" pauses the function until x resolves. Mentally read "await fetch(...)" as "wait for fetch to come back, then continue".

bcrypt

A password-hashing function designed to be slow on purpose. We store `bcrypt("user-password")` instead of the password itself. Comparing on login is also slow, which limits how many guesses an attacker can make per second if the database is leaked.

Brute force

Trying lots of passwords/codes/inputs until one works. Defences: rate limiting, account lockouts, slow hashes, captchas.

CDN

Content Delivery Network — third-party servers that host static assets (Chart.js, fonts) close to the user. Faster than serving everything yourself.

CSP (Content Security Policy)

An HTTP header that tells the browser "only execute scripts from these origins, only fetch from those, only render frames from these". Limits the blast radius of an XSS bug.

CSRF (Cross-Site Request Forgery)

An attack where a malicious site uses your authenticated browser to issue a request to ours. Defence: per-session secret token in a custom header, checked on every state-changing request.

DOM (Document Object Model)

The tree of objects the browser builds from your HTML. JavaScript can read it (`document.getElementById`), modify it (`element.textContent = ...`), or listen for events on it (`button.addEventListener("click", ...)`).

Endpoint

A specific URL on the server. "POST /api/login" is one endpoint; "GET /api/entries" is another. Each one is wired to a handler function.

Express

The Node.js web framework we use. `app.get("/path", handler)` registers a GET endpoint, `app.use(middleware)` wires a middleware into the chain.

Foreign key

A database column whose values must match primary keys in another table. `entries.user_id REFERENCES users(id)` means you cannot insert an entry for a non-existent user.

Helmet

A bundle of Express middlewares that set sensible security HTTP headers (CSP, X-Frame-Options, ...).

HTTP / HTTPS

The protocol browsers and servers speak. HTTPS = HTTP over TLS (encrypted). The "S" is non-negotiable in production.

IIFE (Immediately Invoked Function Expression)

`(function() { ... })()` — a function defined and called in the same expression. Used in this project to give files their own private scope before ES modules were universal.

IV (Initialisation Vector)

A random per-encryption value mixed into the cipher state so identical plaintexts encrypt to different ciphertexts. Stored alongside the ciphertext; never reused.

JSON

JavaScript Object Notation — a simple text format (`{ "key": "value", "list": [1,2,3] }`). The lingua franca of every API in this app.

JWT

JSON Web Token — not used here. We use traditional cookie-based sessions instead.

localStorage

A small per-origin key/value store in the browser. Survives reloads. Used in this project for theme, language, and saved filter state.

Middleware (Express)

A function `(req, res, next) => ...` inserted into the request pipeline. `helmet`, `express.json`, `session`, `CSRF` and `rate limiters` are all middlewares.

Migration

A small SQL script that evolves the schema by adding a column or table. We never rewrite `schema.sql` in place; we ship an idempotent `migrate-add-X.sql` instead.

Multer

Express middleware for handling multipart/form-data — i.e. file uploads.

OAuth

A protocol for delegated access. In this project: Claude Code OAuth tokens (`sk-ant-oat01-...`) and GitHub Copilot OAuth tokens (`gho_...`) authenticate against the provider on behalf of the user. We never see the user's actual login.

Parameterised query

A SQL query with placeholders (`$1`, `$2`, ...) that the driver fills in safely. Defence against SQL injection.

Promise

A JavaScript object representing "a value that will be available later". `async/await` is sugar over Promises.

Rate limit

A cap on how many requests a single client can make in a window. Defends against brute-force, scraping, and accidental loops.

Rejection sampling

A trick for generating uniformly random elements from a set when the underlying source produces too many of them. The `reset-code` generator uses it so every alphabet character is exactly equally

likely.

SQL injection

An attack where user input is concatenated into a SQL string, letting the attacker run arbitrary queries. Fully prevented by parameterised queries.

Session

Server-side state tied to a specific browser via a signed cookie. Stored in the "session" table; persists across server restarts.

Slug

A short, URL-safe identifier ([a-z0-9-]+). Categories use slugs internally so labels can change freely without breaking references.

TOTP

Time-based One-Time Password — the 6-digit codes your authenticator app shows. Standard RFC 6238; we use the `otplib` library.

Transaction (SQL)

BEGIN; ... COMMIT; — a group of statements that either all succeed or all roll back. Used by the JSON migrator and by atomic operations like `upsertUserBudget`.

XSS (Cross-Site Scripting)

An attack where untrusted text is rendered as HTML/JS in the page. Defence: always treat user input as text (`textContent`, not `innerHTML`); use CSP as a second line of defence.

Where to go next

If you have read this whole thing, congratulations — you now have a more complete mental model of this codebase than most people who use it daily. Some suggestions for what to do next:

- Open `server.js` in a real editor (with line numbers visible), keep this PDF open in another window, and walk through the routes you find most interesting line by line.
- Run `npm start` locally, log in, open the browser developer tools (F12 -> Network tab), and watch the JSON traffic on every click. Every request you see will map to one of the endpoints listed in Part IV.
- Skim `db/queries.js` once end-to-end. It is the truest single document of "what data does this app actually have"; once you internalise it, every feature in the front end stops being mysterious.
- When you want to add a feature, start by writing the SQL query first (in `db/queries.js`), then the route in `server.js`, then the UI in `index.html` + `app.js`. That order works because each layer constrains the next.

Happy hacking.